

Lab Sheet-1

Ashish singh Rawat

Cu24250221

```
import pandas as pd

import numpy as np

import matplotlib.pyplot as plt

import seaborn as sns

from sklearn.preprocessing import LabelEncoder, StandardScaler

# Set aesthetic styling

sns.set_theme(style="whitegrid")

# Step 1, 2 & 3: Load the Dataset and Inspect

# Using 'HR Employee-Attrition.csv'

filename = "HR Employee-Attrition.csv"

df = pd.read_csv(filename)

print("=" * 60)

print("INITIAL DATA INSPECTION")

print("=" * 60)

print(f"Dataset Shape: {df.shape}")

print("\nFirst 5 Rows:")

print(df.head())

print("\nData Info:")

print(df.info())

print("\nDescriptive Summary:")

print(df.describe().T[['count', 'mean', 'std', 'min', '50%', 'max']])

# Step 4: Missing Values Identification & Handling

print("\n" + "=" * 60)

print("MISSING VALUES ANALYSIS")

print("=" * 60)

missing_counts = df.isnull().sum()

print(missing_counts[missing_counts > 0] if missing_counts.sum() > 0 else "No missing values found.")

# Industry-standard imputation pipeline for numerical/categorical nulls

for col in df.columns:

    if df[col].isnull().sum() > 0:
```

```

    if df[col].dtype in ['int64', 'float64']:
        df[col] = df[col].fillna(df[col].median())
    else:
        df[col] = df[col].fillna(df[col].mode()[0])

# Step 5: Duplicate Detection & Removal
print("\n" + "=" * 60)
print("DUPLICATE RECORD CHECK")
print("=" * 60)

duplicate_count = df.duplicated().sum()
print(f"Number of duplicate rows identified: {duplicate_count}")

if duplicate_count > 0:
    df.drop_duplicates(inplace=True)
    print("Duplicates removed successfully.")

# Drop non-informative constant or ID columns standard in this dataset
cols_to_drop = ['EmployeeCount', 'StandardHours', 'Over18', 'EmployeeNumber']
df.drop(columns=[col for col in cols_to_drop if col in df.columns], inplace=True)
print(f"Removed non-informative constant/identifier columns: {cols_to_drop}")

# Step 6: Categorical Encoding

# Binary categorical columns -> Label Encoding
binary_cols = ['Attrition', 'OverTime', 'Gender']

le = LabelEncoder()

for col in binary_cols:
    if col in df.columns:
        df[col] = le.fit_transform(df[col])

# Multi-class nominal columns -> One-Hot Encoding
nominal_cols = ['BusinessTravel', 'Department', 'EducationField', 'JobRole', 'MaritalStatus']

df = pd.get_dummies(df, columns=[col for col in nominal_cols if col in df.columns], drop_first=True)

print("\nCategorical variables successfully converted via Label and One-Hot Encoding.")

# Step 7: Outlier Detection and IQR Capping (Winsorization)

plt.figure(figsize=(6, 4))

sns.boxplot(x=df['MonthlyIncome'], color='#3498db')

```

```

plt.title("Boxplot of Monthly Income (Before IQR Capping)", fontsize=12, weight='bold')

plt.savefig("exp1_boxplot_outliers.png", dpi=300, bbox_inches='tight')

plt.close()

print("[SAVED] exp1_boxplot_outliers.png")

# IQR Capping for MonthlyIncome

Q1 = df['MonthlyIncome'].quantile(0.25)
Q3 = df['MonthlyIncome'].quantile(0.75)

IQR = Q3 - Q1

lower_bound = Q1 - 1.5 * IQR
upper_bound = Q3 + 1.5 * IQR

outlier_count = ((df['MonthlyIncome'] < lower_bound) | (df['MonthlyIncome'] > upper_bound)).sum()

print(f"MonthlyIncome Outliers detected: {outlier_count}")

# Cap outliers to theoretical boundaries

df['MonthlyIncome'] = np.clip(df['MonthlyIncome'], lower_bound, upper_bound)

print("Outliers capped using IQR threshold boundaries.")

# Step 8: Normalization / Standardization

scaler = StandardScaler()

scale_features = ['Age', 'MonthlyIncome', 'TotalWorkingYears', 'YearsAtCompany']

scaled_cols = [f"{col}_Scaled" for col in scale_features]

df[scaled_cols] = scaler.fit_transform(df[scale_features])

print("\nNumerical features standardized using StandardScaler (mean=0, std=1).")

# Step 9: Feature Engineering

# Create discrete Age Group categories

df['AgeGroup'] = pd.cut(

    df['Age'],

    bins=[17, 30, 45, 65],

    labels=['Young', 'Middle-Aged', 'Senior']

)

# Create Tenure Stability Ratio (Years at company relative to total working years)

df['TenureRatio'] = df['YearsAtCompany'] / (df['TotalWorkingYears'] + 1)

# Categorize Income tiers

```

```

df['IncomeCategory'] = pd.qcut(

    df['MonthlyIncome'],

    q=3,

    labels=['Low', 'Medium', 'High']

)

print("Engineered new attributes: AgeGroup, TenureRatio, IncomeCategory.")

# Step 10: Export Preprocessed Dataset

output_filename = "Cleaned_HR_Employee_Attrition.csv"

df.to_csv(output_filename, index=False)

print(f"\nPreprocessed dataset exported successfully to '{output_filename}'.")

print(f"Final Cleaned Dataset Dimensions: {df.shape}")

```

```

INITIAL DATA INSPECTION
=====
Dataset Shape: (1470, 35)

First 5 Rows:
   Age  Attrition  ...  YearsSinceLastPromotion  YearsWithCurrManager
0   41         Yes  ...                      0                      5
1   49         No  ...                      1                      7
2   37         Yes  ...                      0                      0
3   33         No  ...                      3                      0
4   27         No  ...                      2                      2

[5 rows x 35 columns]

Data Info:
<class 'pandas.DataFrame'>
RangeIndex: 1470 entries, 0 to 1469
Data columns (total 35 columns):
#   Column                Non-Null Count  Dtype
---  -
0   Age                    1470 non-null  int64
1   Attrition              1470 non-null  str
2   BusinessTravel         1470 non-null  str
3   DailyRate              1470 non-null  int64
4   Department             1470 non-null  str
5   DistanceFromHome       1470 non-null  int64
6   Education              1470 non-null  int64
7   EducationField          1470 non-null  str
8   EmployeeCount          1470 non-null  int64
9   EmployeeNumber         1470 non-null  int64
10  EnvironmentSatisfaction  1470 non-null  int64
11  Gender                 1470 non-null  str
12  HourlyRate             1470 non-null  int64
13  JobInvolvement          1470 non-null  int64
14  JobLevel               1470 non-null  int64
15  JobRole                1470 non-null  str
16  JobSatisfaction         1470 non-null  int64

```

```

Descriptive Summary:

```

	count	mean	...	50%	max
Age	1470.0	36.923810	...	36.0	60.0
DailyRate	1470.0	802.485714	...	802.0	1499.0
DistanceFromHome	1470.0	9.192517	...	7.0	29.0
Education	1470.0	2.912925	...	3.0	5.0
EmployeeCount	1470.0	1.000000	...	1.0	1.0
EmployeeNumber	1470.0	1024.865306	...	1020.5	2068.0
EnvironmentSatisfaction	1470.0	2.721769	...	3.0	4.0
HourlyRate	1470.0	65.891156	...	66.0	100.0
JobInvolvement	1470.0	2.729932	...	3.0	4.0
JobLevel	1470.0	2.063946	...	2.0	5.0
JobSatisfaction	1470.0	2.728571	...	3.0	4.0
MonthlyIncome	1470.0	6502.931293	...	4919.0	19999.0
MonthlyRate	1470.0	14313.103401	...	14235.5	26999.0
NumCompaniesWorked	1470.0	2.693197	...	2.0	9.0
PercentSalaryHike	1470.0	15.209524	...	14.0	25.0
PerformanceRating	1470.0	3.153741	...	3.0	4.0
RelationshipSatisfaction	1470.0	2.712245	...	3.0	4.0
StandardHours	1470.0	80.000000	...	80.0	80.0
StockOptionLevel	1470.0	0.793878	...	1.0	3.0
TotalWorkingYears	1470.0	11.279592	...	10.0	40.0
TrainingTimesLastYear	1470.0	2.799320	...	3.0	6.0
WorkLifeBalance	1470.0	2.761224	...	3.0	4.0
YearsAtCompany	1470.0	7.008163	...	5.0	40.0
YearsInCurrentRole	1470.0	4.229252	...	3.0	18.0
YearsSinceLastPromotion	1470.0	2.187755	...	1.0	15.0
YearsWithCurrManager	1470.0	4.123129	...	3.0	17.0

```

[26 rows x 6 columns]

=====
MISSING VALUES ANALYSIS
=====
No missing values found.

```

